

# 파이썬 프로그래밍 2주차 요약본

## 6. 함수1

### 6.1 함수 정의

- **내장(Built-in) 함수**
  - 파이썬에 포함되어 있는 함수
  - 매개변수 (Parameter) : 함수에 전달하는 입력 값
  - 반환값 : 함수 실행 결과로 돌려주는 값
- **모듈(Module)의 함수**
  - 비슷한 함수끼리 모아둔 파일
- **사용자 정의 함수**
  - 직접 만들어 사용하는 함수

```
def my_func(): # def 함수이름(매개변수):  
    print("토끼야 안녕!") # 명령어  
    # [return 반환값]  
  
my_func() # 함수 이름(인수)
```

### 6.2 함수 호출

- 정의된 함수의 기능을 사용
- 함수명을 사용할 인수와 함께 사용

### 6.3 매개변수와 함수 호출

- 매개변수가 없는 함수 호출

```
def fhello():  
    print("매개변수가 없는 함수 호출하기")
```

```
fhello()
```

- 매개변수가 있는 함수 호출

```
def funca(na, nb):  
    nc = na+nb  
    print(na,"+",nb,"=",nc)  
  
funca(10, 20)
```

## 6.4 return 키워드

- 반환값 : 함수 실행 결과로 돌려주는 값(r\_value)

## 6.5 함수 호출 순서

- 함수는 호출 되기 전에 정의되어야 함

```
def fadd(pa, pb):  
    pc = pa+pb  
    return pc  
  
na = 10  
nb = 20  
nc = fadd(na, nb)  
print(na,"+",nb,"결괏값은",nc,"이다.")
```

## 6.6 함수 반환값이 없을 때

```
def fplusminus(arg):  
    if arg > 0:  
        return "plus"  
    elif arg < 0:  
        return "minus"  
  
stra = fplusminus(0) # 인수가 0인 경우 반환할 값이 없음  
print(stra) # 이 경우 함수는 호출자에게 None 을 반환
```

## 7. 함수2

### 7.1 SWAP

- 두 데이터 서로 맞바꾸기

```
def funca(pa, pb):  
    temp = pa  
    pa = pb  
    pb = temp  
  
na = 10  
nb = 11  
print("na 값은", na, "nb 값은 ", nb)  
funca(na, nb)  
print("na 값은", na, "nb 값은 ", nb)
```

- 데이터 타입: 정수형, 실수형, 문자열, 리스트, 튜플, 딕셔너리 등.

### 7.2 전역 변수

- 네임스페이스(namespace, 이름공간): 특정 객체를 이름에 따라 구분 가능한 범위
- 전역변수(Global Variable)
  - 프로그램 전체에서 접근 가능한 유효 범위를 갖는 변수
  - 함수 외부에서 정의된 변수

```
b = 2 # 전역변수  
  
def scope_test():  
    a = 1
```

### 7.3 지역 변수

- 지역 변수(Local Variable)
  - 해당 변수가 생성된 범위(코드 블록) 내부에서만 유효

- 함수 내부에서 정의된 변수

```
def scope_test():
    a = 1 # 지역변수
```

## 7.4 global 키워드

- **global:** 변수 유효범위(변수 스코프, Variable Scope)
  - namespace에 정의되어 있는 접근 가능 범위

```
def scope_test():
    global a # a의 지역 변수 생성 차단, 전역 변수 a 사용 선언
    a = 1

scope_test()
print(a)
```

## 7.5 디폴트 매개변수

- 일반적으로 함수에서 매개변수와 인수는 일대일 대응 관계

```
def personb(width, height):
    print("함수디폴트값없음 width=", width, "height=", height)
```

## 7.6 함수의 디폴트 매개변수

```
def personb(width=4, height=3):
    print("함수디폴트값설정 width=", width, "height=", height)
```

# 8. 자료구조와 알고리즘

## 8.1 리스트 데이터 SWAP

- 리스트 데이터 값 서로 바꾸기

```
ca = [10, 11]
temp = ca[0]
ca[0] = ca[1]
ca[1] = temp
```

## 8.2 id 함수와 리스트

- 리스트 메모리 활용

```
a = [10, 11, 12, 13]
b = a
```

## 8.3 최대값 찾기(1)

- 주어진 리스트에서 최대값을 구하는 프로그램 작성하기

```
ca = [10, 17, 13, 11]
max = ca[0]
for sb in ca:
    if max < sb:
        max = sb
print(max)
```

## 8.4 선택 정렬

- 주어진 리스트 요소 중 최소값을 찾아 첫 번째 위치의 요소와 서로 교환하는 과정을 모든 요소가 정렬될 때까지 반복하는 알고리즘

```
ca = [21, 10, 11, 15, 13]
mina = ca[0]
minix = 0
for sb in range(1, 5, 1):
    if mina > ca[sb]:
        mina = ca[sb]
        minix = sb

temp = ca[0]
ca[0] = ca[minix]
```

```
ca[minix] = temp
print(ca)
```

## 8.5 최대값 찾기(2)

- 최대값 반환 프로그램 작성

```
su = [5, 4, 7, 10, 6]

def fmax(a, b, c, d, e):
    max = a
    if max < b:
        max = b
    if max < c:
        max = c
    if max < d:
        max = d
    if max < e:
        max = e
    return max

a = su[0]
b = su[1]
c = su[2]
d = su[3]
e = su[4]

max = fmax(su[0], su[1], su[2], su[3], su[4])
```

## 8.6 딕셔너리를 매개변수로 사용

- 딕셔너리 데이터 할당에 따른 값 변화 확인

```
a = {1: "월", "tue": "화", "wed": "수"}
print(a)
x = a
```

```
print(x)
x[1] = "일"
print(x)
print(a)
```

## 9. 클래스1

### 9.1 클래스 구성 및 접근

- **절차적 프로그래밍(Procedural Programming)**: 작업의 기능적/순차적인 처리에 중점을 둔 프로그래밍
- **객체 지향 프로그래밍(Object Oriented Programming)**: 실제 세계의 객체를 모델로 프로그래밍
- **클래스 (class)**: 분류(종류에 따라 가르는 것), 틀, 설계도
- **객체 (object)**: 분류에 따라 구분되는 실체, 실제 존재하는 것(클래스 선언된 식별자)
- **인스턴스 (instance)**: 클래스를 통해 객체가 실제로 구현된 예시(메모리 적재 시)

### 9.2 클래스(class)와 인스턴스

- **객체 생성**: 클래스를 통해 객체 생성

```
na = PizzaClass() #객체 이름 = 클래스 이름(인수)
```

### 9.3 객체의 멤버 접근

- 객체 멤버(멤버변수, 멤버함수)의 활용을 위해 접근연산자(.) 이용

```
class Singer:
    name = "아이유"
    def sing(self):
        print("이 밤 그날의 반딧불을 당신의 창 가까이 보낼게요~")

iu = Singer()
```

```
print(iu.name)
iu.sing()
```

- **클래스 멤버(변수)**: 클래스 내, 클래스 메소드 외부에서 선언
- **인스턴스 멤버(변수)**: 클래스 내, 클래스 메소드 내부에서 선언

## 9.4 self

- 해당 클래스의 객체를 의미(해당 객체의 저장 위치 정보 포함)

```
class Bag:
    def __init__(self):
        self.data = []

    def add(self, x):
        self.data.append(x)

    def addtwice(self, x):
        self.add(x)
        self.add(x)
```

## 9.5 init() 초기화 메서드

- **생성자**: 객체 생성 메서드

```
class Human:
    def __init__(self, age, name):
        self.age = age
        self.name = name
    def intro(self):
        print(str(self.age) + "살" + self.name + "입니다.")

kim = Human(29, "김상형")
kim.intro()
```

```
lee = Human(45, "이승우")
lee.intro()
```

## 9.6 내장 클래스

- 파이썬 내부에 기 정의된 클래스
- 파이썬에서 정의한 클래스(파이썬에서 제공하는 데이터형) : int, float, str, list, dictionary

```
strb = str("abc")
print(strb)
strc = strb.capitalize()
print(strc)
```

## 10. 클래스2

### 10.1 모듈 생성

- 모듈: 클래스, 함수, 변수 등을 모아둔 파일

```
# mex1.py
class Cvalue:
    def __init__(self):
        self.lista = []
    def add(self.num):
        self.lista.append(num)
    def fprint(self):
        print(self.lista)

def plus(a, b):
    c = a + b
    return c

p1 = Cvalue()
p1.add(1)
```

```
p1.add(2)
p1.add(3)
p1.fprint()
```

## 10.2 import

- 불러온 모듈 뒤에 접근연산자(.)를 붙여 해당 모듈이 포함한 내용 사용

```
import mex1

p2 = mex1.Cvalue()
p2.add(11)
p2.add(21)
p2.add(31)
p2.fprint()

value = mex1.plus(10, 20)
print(value)
```

## 10.3 from 모듈명 import 함수

- 모듈(라이브러리)을 import 시, 전체 모듈이 아닌 필요한 부분만 사용 가능

```
from mex1 import plus # from 모듈 이름 import 함수
```

- import \* 형식으로 모듈 이름과 접근 연산자 없이 모든 내용을 사용 가능

```
from mex1 import * # from 모듈 이름 import *
```

## 10.4 메인 모듈과 하위 모듈

- 메인 모듈 : 제일 먼저 실행되는 모듈

```
# mex5.py
from mex4 import plus

value = plus(10, 20)
print(value)
```

- **하위 모듈** : 메인 모듈이 import 문을 이용하여 불러오는 모듈

```
# mex4.py
class Cvalue:
    def __init__(self):
        self.lista = []
    def add(self.num):
        self.lista.append(num)
    def fprint(self):
        print(self.lista)

def plus(a, b):
    c = a + b
    return c

p1 = Cvalue()
p1.add(1)
p1.add(2)
p1.add(3)
p1.fprint()
```

## 10.5 상속

- 어떤 클래스가 다른 클래스의 성질을 물려받는 것
  - 부모클래스(super class)

```
class Human:
    eyes = 2
    nose = 1
    mouth = 1
```

```

def __init__(self, name, age):
    self.name = name
    self.age = age

def intro(self):
    print(f"이름:{self.name}")
    print(f"나이:{self.age}")

def eat(self):
    print("먹다")

def sleep(self):
    print("자다")

def talk(self):
    print("말하다")

```

- 자식클래스(sub class)

```

class Student(Human):
    def study(self):
        print("공부하다")

```

## 10.6 오버라이딩

- 부모 클래스의 메소드를 동일한 이름으로 자식 클래스에서 재정의하여 사용하는 것

```

class Animal:
    def eat(self):
        print("먹다")
    def move(self):
        print("이동하다")
    def sound(self):
        print("소리내다")

class Cat(Animal):

```

```

def eat(self):
    print("잡식하다")
def sound(self):
    print("야옹~", end="")
    super().sound()

cat1 = Cat()
cat1.eat()
cat1.move()
cat1.sound()

```

- **super():** 자식 클래스에서 부모의 메서드 호출할 때 사용

```

class Student(Human):
    def __init__(self, name, age, studentNum):
        super().__init__(name, age)
        self.studentNum = studentNum

    def introduce(self):
        super().introbude()
        print(f"학번:{self.studentNum}")

    def study(self):
        print("공부하다")

hong = Human("홍길동", 33)
hong.introduce()
kim = Student("김유빈", 20, 20220523)
kim.introduce()
kim.study()

```